



МИНОБРНАУКИ РОССИИ  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«МИРЭА – Российский технологический университет»

**РТУ МИРЭА**

---

Институт информационных технологий (ИИТ)  
Кафедра математического обеспечения и стандартизации информационных технологий  
(МОСИТ)

## **ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ**

по дисциплине «Тестирование и верификация программного обеспечения»

### **Практическое занятие № 2**

*ИКБО-43-23 ОГАНЕСЯН Д. К.  
КОЩЕЕВ М.И.  
ГУТКОВИЧ А.А.  
ИВАНОВ А.В.  
УЛЗЕТУЕВ А.С.*

---

(подпись)

**АССИСТЕНТ**

*ИЛЬИЧЕВ Г.П.*

---

(подпись)

**ОТЧЕТ  
ПРЕДСТАВЛЕН**

**«17»\_\_ОКТЯБРЯ\_2025 Г.**

Москва 2025 г.

## СОДЕРЖАНИЕ

|                                                |    |
|------------------------------------------------|----|
| 1 ЦЕЛЬ И ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ.....       | 3  |
| 2 ПРАКТИЧЕСКАЯ ЧАСТЬ.....                      | 4  |
| 2.1 Разработка модулей.....                    | 4  |
| 2.2 Модульное тестирование.....                | 13 |
| 2.3 Мутационное тестирование.....              | 33 |
| 3 АНАЛИЗ И ВЫВОДЫ.....                         | 53 |
| 3.1 Оценка качества тестирования.....          | 53 |
| 3.2 Анализ недостатков и их устранение.....    | 53 |
| 3.3 Итоговые выводы по результатам работы..... | 54 |

## 1 ЦЕЛЬ И ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ

Цель работы: познакомить студентов с процессом модульного и мутационного тестирования, включая разработку, проведение тестов, исправление ошибок, анализ тестового покрытия, а также оценку эффективности тестов путём применения методов мутационного тестирования.

Для достижения поставленной цели работы студентам необходимо выполнить ряд задач:

- изучить основы модульного тестирования и его основные принципы;
- освоить использование инструментов для модульного тестирования (pytest для Python, JUnit для Java и др.);
- разработать модульные тесты для программного продукта и проанализировать их покрытие кода;
- изучить основы мутационного тестирования и освоить инструменты для его выполнения (MutPy, PIT, Stryker);
- применить мутационное тестирование к программному продукту, оценить эффективность тестов;
- улучшить существующий набор тестов, ориентируясь на результаты мутационного тестирования;
- оформить итоговый отчёт с результатами проделанной работы.

## 2 ПРАКТИЧЕСКАЯ ЧАСТЬ

### 2.1 Разработка модулей

#### а) Модуль «convert»

Описание функциональности: Модуль представляет собой систему конвертации единиц измерения, включающую словарь базовых единиц для длины (миллиметры, сантиметры, метры, километры) и массы (граммы, килограммы, центнеры, тонны), а также набор функций для преобразования температур между шкалами Цельсия, Фаренгейта и Кельвина. Основной функционал реализован через функцию `convert_linear` для конвертации линейных единиц и специализированные функции для температурных преобразований, при этом система обеспечивает валидацию входных данных и корректное математическое преобразование значений с использованием коэффициентов масштабирования.

*Листинг 1 — Исходный код.*

```
# Единицы измерения
UNITS = {
    "длина": {
        "миллиметр": 10 ** -3, # 0.001 метра
        "сантиметр": 10 ** -2, # 0.01 метра
        "дециметр": 10 ** -1, # 0.1 метра
        "метр": 1, # базовая единица
        "километр": 10 ** 3 # 1000 метров
    },
    "масса": {
        "грамм": 1, # базовая единица
        "килограмм": 10 ** 3, # 1000 грамм
        "центнер": 10 ** 5, # 100 000 грамм
        "тонна": 10 ** 6 # 1 000 000 грамм
    }
}

# Функция преобразования цельсия в фаренгейты
def c_to_f(c):
    return c * 9.0 + 32.0

# Функция преобразования фаренгейтов в цельсии
def f_to_c(f):
    return (f - 32.0) * 5.0 / 9.0

# Функция преобразования цельсия в кельвины
def c_to_k(c):
```

```

    return c + 273.15

# Функция преобразования цельвинов в цельсии
def k_to_c(k):
    return k - 273.15

# Функция преобразования фааренгейтов в кельвины
def f_to_k(f):
    return (f - 32.0) * 5.0 / 9.0 + 273.15

# Функция преобразования кельвинов в фаренгейты
def k_to_f(k):
    return (k - 273.15) * 9.0 / 5.0 + 32.0

# Функция преобразования остальных категорий
def convert_linear(category, value, from_unit, to_unit):
    if category not in UNITS:
        raise ValueError("Неизвестная категория конверсии: " + str(category))

    units = UNITS[category]
    if from_unit not in units:
        raise ValueError("Неизвестная единица: " + from_unit)
    if to_unit not in units:
        raise ValueError("Неизвестная единица: " + to_unit)
    base_value = value * units[from_unit]
    result = base_value / units[to_unit]
    return result

```

## б) Модуль «history»

Описание функциональности: Модуль представляет собой систему ведения истории конвертаций, которая позволяет загружать и сохранять данные в JSON-файл, добавлять новые записи с указанием категории, исходных и конечных единиц измерения, результата и временной метки, получать последние конвертации, фильтровать их по категориям, очищать всю историю и формировать статистику использования (общее количество конвертаций, распределение по категориям, даты первой и последней операции). Все операции с данными осуществляются через работу с файлом `conversion_history.json`, при этом предусмотрена возможность включения режима отладки для дополнительной информации о выполняемых операциях.

*Листинг 2 — Исходный код.*

```

import json
import datetime
from pathlib import Path

debug = False

```

```

FILENAME = Path("conversion_history.json")

def load_history():
    #Загрузить историю из файла
    if FILENAME.exists():
        with open(FILENAME, 'r', encoding='utf-8') as f:
            return json.load(f)
    return []

def save_history(history):
    #Сохранить историю в файл
    with open(FILENAME, 'w', encoding='utf-8') as f:
        json.dump(history, f, ensure_ascii=False, indent=2)
        print("Json has been saved.")
    if debug:
        print("-----")
        print(f"history: {history}")
        print("-----")

def add_conversion(category, value, from_unit, to_unit, result):
    #Добавить запись о конвертации
    history = load_history()
    entry = {
        "timestamp": datetime.datetime.now().isoformat(),
        "category": category,
        "value": value,
        "from_unit": from_unit,
        "to_unit": to_unit,
        "result": result
    }
    history.append(entry)
    save_history(history)

def get_recent_conversions(count=5):
    #Получить последние сколько-то конвертаций
    history = load_history()
    return history[count:]

def clear_history():
    #Очистить всю историю
    save_history([])

def get_conversions_by_category(category):
    #Получить все конвертации по категории
    history = load_history()
    return [entry for entry in history if entry["category"] == category]

def get_statistics():
    #Получить статистику по истории
    history = load_history()
    if not history:
        return {"total": 0}

    categories = {}
    for entry in history:
        cat = entry["category"]
        categories[cat] = categories.get(cat, 0) + 1

    return {

```

```
"total": len(history),
"by_category": categories,
"first_conversion": history[0]["timestamp"],
"last_conversion": history[-1]["timestamp"]
}
```

### в) Модуль «unit\_formater»

Описание функциональности: Модуль предоставляет набор функций для форматирования результатов конвертации и истории операций: он форматирует числовые значения с заданной точностью, включая научную нотацию для очень больших или малых чисел; корректно склоняет названия единиц измерения в зависимости от числового значения; обрабатывает специальные символы для температурных шкал (°C, °F, K); формирует полное представление результата конвертации с исходными и конечными единицами измерения; форматирует записи из истории конвертаций, включая дату, время и результат преобразования в удобочитаемом виде.

*Листинг 3 — Исходный код.*

```
import datetime

# Функция форматирования числа с заданной точностью
def format_number(value, precision=6):
    if value == 0:
        return "0"

    if abs(value) >= 1e6 or (abs(value) <= 1e-4 and value != 0):
        return f"{value:.{precision}e}"

    formatted = f"{value:.{precision}f}"
    if '.' in formatted:
        formatted = formatted.rstrip('0').rstrip('.')

    return formatted

# Функция форматирования названия единиц измерения
def format_unit_name(unit, value):
    if value == 1:
        return unit

    plural_rules = {
        'метр': 'метры',
        'километр': 'километры',
        'сантиметр': 'сантиметры',
        'грамм': 'граммы',
        'килограмм': 'килограммы',
        'тонна': 'тонны'
    }

    return plural_rules.get(unit, unit)
```

```

# Функция специального форматирования для температур
def format_temperature(value, from_unit, to_unit):
    symbols = {
        'c': '°C',
        'f': '°F',
        'k': 'K',
        'цельсий': '°C',
        'фаренгейт': '°F',
        'кельвин': 'K'
    }

    from_symbol = symbols.get(from_unit.lower(), from_unit)
    to_symbol = symbols.get(to_unit.lower(), to_unit)

    return from_symbol, to_symbol

# Функция форматирования полного результата конвертации
def format_conversion_result(value, from_unit, result, to_unit):
    formatted_value = format_number(value)
    formatted_result = format_number(result)

    if any(unit in ['c', 'f', 'k', 'цельсий', 'фаренгейт', 'кельвин']
           for unit in [from_unit, to_unit]):
        from_sym, to_sym = format_temperature(value, from_unit, to_unit)
        return f"{formatted_value}{from_sym} = {formatted_result}{to_sym}"

    from_name = format_unit_name(from_unit, value)
    to_name = format_unit_name(to_unit, result)

    return f"{formatted_value} {from_name} = {formatted_result} {to_name}"

# Функция форматирования записи истории для красивого вывода
def format_history_entry(entry):
    timestamp = datetime.datetime.fromisoformat(entry['timestamp'])
    date_str = timestamp.strftime("%d.%m.%Y %H:%M")
    result_str = format_conversion_result(
        entry['value'],
        entry['from_unit'],
        entry['result'],
        entry['to_unit']
    )
    return f"[{date_str}] {result_str}"

```

### г) Модуль «validator»

Описание функциональности: Модуль содержит набор функций для валидации входных данных при конвертации: проверяет, является ли введенное значение корректным числом; убеждается, что число положительное; контролирует, находится ли температура в допустимом диапазоне для выбранной шкалы (Цельсия, Фаренгейта или Кельвина); проверяет существование указанной единицы измерения в заданной категории; а также



осуществляет проверку наличия самой категории в системе единиц измерения, при этом в случае обнаружения ошибок генерирует соответствующие сообщения об ошибках.

*Листинг 4 — Исходный код.*

```
# Функция проверки является ли числом
def validate_numeric_input(value_str):
    try:
        return float(value_str)
    except ValueError:
        raise ValueError("Ошибка: введите корректное число")

# Функция проверки что число положительное
def validate_positive_number(value):
    if value <= 0:
        raise ValueError("Ошибка: значение должно быть положительным")
    return value

# Функция проверки допустимости температуры
def validate_temperature_range(value, scale):
    ranges = {
        'c': (-273.15, 10000),
        'f': (-459.67, 18032),
        'k': (0, 10273.15)
    }
    min_temp, max_temp = ranges.get(scale, (-float('inf'), float('inf')))
    if not min_temp <= value <= max_temp:
        raise ValueError(f"Температура должна быть в диапазоне {min_temp} - {max_temp}")
    return value

# Функция проверки существования единицы измерения
def validate_unit_exists(category, unit):
    from core.convert_script import UNITS
    if category not in UNITS:
        raise ValueError(f"Категория '{category}' не найдена")
    if unit not in UNITS[category]:
        raise ValueError(f"Единица '{unit}' не найдена в категории '{category}'")
    return True

# Функция проверки существования категории
def validate_category(category):
    from core.convert_script import UNITS
    if category not in UNITS:
        raise ValueError(f"Категория '{category}' не найдена")
    return True
```

#### **д) Модуль «view»**

Описание функциональности: Модуль представляет собой консольное приложение для конвертации единиц измерения, которое включает главное меню с возможностью конвертации длины, массы и температуры, просмотра истории операций и статистики; предоставляет пользователю интерфейс для

выбора типа конвертации, ввода значений и единиц измерения, отображает результаты преобразований и сохраняет их в историю; содержит подменю для работы с историей (просмотр последних конвертаций, всей истории, статистики и очистка), а также механизмы валидации входных данных и обработки ошибок при выполнении конвертаций.

Листинг 5 — Исходный код.

```
import time
import core.convert_script as conversions
import core.validator as validator
import core.unit_formatter as unit_formatter
import core.history as history

def startup():
    print(r"""
/$$$$\  $$$$$\ $$\  $$\ $$$\  $$$\ $$$$$$$\ $$$$$$\ $$$$$$$\ $$$$$\ $$$
$$$$\
$$ _$$\ $$ _$$\ $$$\ $$ |$$ |  $$ |$$ _||$$ _$$\__$$ _||$$ _||$$
_$$\
$$ /  \_||$$ /  $$ |$$$$\ $$ |$$ |  $$ |$$ |  $$ |  $$ |  $$ |  $$
|  $$ |
$$ |  $$ |  $$ |$$ $$$\ $$ |\$$\  $$ |$$$$$\  $$$$$$  |  $$ |  $$$$$\  $$
$$$$$  |
$$ |  $$ |  $$ |$$ \$$$$ | \$$\$$ /  $$ _|  $$ _$$<  $$ |  $$ _|  $$
_$$<
$$ |  $$\ $$ |  $$ |$$ |\$$$ | \$$$ /  $$ |  $$ |  $$ |  $$ |  $$
|  $$ |
\$$$$$  | $$$$$$ |$$ | \$$ |  \$ /  $$$$$$$\ $$ |  $$ |  $$ |  $$$$$$$\ $$
|  $$ |
 \_||  \_||  \_||  \_||  \_||  \_||  \_||  \_||  \_||  \_||  \_||
_||  \_||
""")
    time.sleep(1)
    while True:
        menu()

#Главное меню
def menu():
    print()
    print("Что вы хотите сделать?")
    print("1. Конвертировать")
    print("2. Посмотреть историю")
    print("0. Выйти")

    try:
        choice = int(input(">> "))
        if choice == 1:
            convert_menu()
        elif choice == 2:
            history_menu()
        elif choice == 0:
            exit()
        else:
            print("Неверный ввод!")
    except ValueError:
        print("Введите число!")
```

```

#Меню истории
def history_menu():
    while True:
        print("\nИстория конвертаций:")
        print("1. Последние 5 конвертаций")
        print("2. Вся история")
        print("3. Статистика")
        print("4. Очистить историю")
        print("0. Назад")
        choice = input(">> ")
        if choice == "1":
            show_recent_history()
        elif choice == "2":
            show_full_history()
        elif choice == "3":
            show_statistics()
        elif choice == "4":
            history.clear_history()
            print("История очищена.")
        elif choice == "0":
            return
        else:
            print("Неверный ввод!")

def show_recent_history():
    data = history.get_recent_conversions(5)
    if not data:
        print("История пуста.")
        return
    print("\nПоследние 5 конвертаций:")
    for entry in data:
        print(
            f"[{entry['timestamp']}] {entry['category']}: {entry['value']} "
            f"{entry['from_unit']} → {entry['result']} {entry['to_unit']}")

def show_full_history():
    data = history.load_history()
    if not data:
        print("История пуста.")
        return
    print("\nВся история:")
    for entry in data:
        print(
            f"[{entry['timestamp']}] {entry['category']}: {entry['value']} "
            f"{entry['from_unit']} → {entry['result']} {entry['to_unit']}")

def show_statistics():
    stats = history.get_statistics()
    if stats["total"] == 0:
        print("История пуста.")
        return
    print("\nСтатистика:")
    print(f"Всего конвертаций: {stats['total']}")
    print("По категориям:")
    for cat, count in stats["by_category"].items():
        print(f"- {cat}: {count}")
    print(f"Первая: {stats['first_conversion']}")
    print(f"Последняя: {stats['last_conversion']}")

```

```

#Меню выбора типа конвертации
def convert_menu():
    print()
    print("Что вы хотите конвертировать?")
    print("1. Длина")
    print("2. Температура")
    print("3. Масса")
    print("0. Назад")

    choice = input(">> ")
    if choice == "1":
        convert_linear_menu("длина")
    elif choice == "2":
        convert_temperature_menu()
    elif choice == "3":
        convert_linear_menu("масса")
    elif choice == "0":
        return
    else:
        print("Неверный ввод!")

#Конвертации длины/массы
def convert_linear_menu(category):
    print("\nДоступные единицы:")
    for unit in conversions.UNITS[category]:
        print(" -", unit)

    try:
        value_str = input("\nВведите значение: ")
        value = validator.validate_numeric_input(value_str)
        value = validator.validate_positive_number(value)

        from_unit = input("Из (например, метр): ").strip().lower()
        to_unit = input("В (например, километр): ").strip().lower()

        validator.validate_unit_exists(category, from_unit)
        validator.validate_unit_exists(category, to_unit)

        result = conversions.convert_linear(category, value, from_unit, to_unit)
        formatted_result = unit_formatter.format_conversion_result(value, from_unit,
result, to_unit)
        print(f"\n{formatted_result}\n")

        history.add_conversion(category, value, from_unit, to_unit, result)

    except Exception as e:
        print("Ошибка:", e)

#Конвертации температур
def convert_temperature_menu():
    print("""
1. Цельсий --> Фаренгейт
2. Фаренгейт --> Цельсий
3. Цельсий --> Кельвин
4. Кельвин --> Цельсий
5. Фаренгейт --> Кельвин
6. Кельвин --> Фаренгейт
0. Назад
""")
    try:

```

```

str_choice = input(">> ")
choice = validator.validate_numeric_input(str_choice)

if choice == 0:
    return

str_value = input("Введите значение: ")
value = validator.validate_numeric_input(str_value)

conversions_map = {
    1: ("цельсий", "фаренгейт", conversions.f_to_c),
    2: ("фаренгейт", "цельсий", conversions.c_to_f),
    3: ("цельсий", "кельвин", conversions.c_to_k),
    4: ("кельвин", "цельсий", conversions.k_to_c),
    5: ("фаренгейт", "кельвин", conversions.f_to_k),
    6: ("кельвин", "фаренгейт", conversions.k_to_f)
}

if choice not in conversions_map:
    print("Неверный выбор!")
    return

from_unit, to_unit, func = conversions_map[choice]
value = validator.validate_temperature_range(value, from_unit[0])
result = func(value)

formatted_result = unit_formatter.format_conversion_result(value, from_unit,
result, to_unit)
print(f"\n{formatted_result}\n")

history.add_conversion("температура", value, from_unit, to_unit, result)

except ValueError:
    print("Ошибка: введите число!")

if __name__ == "__main__":
    startup()

```

## 2.2 Модульное тестирование

### а) Модуль «convert»

Описание тестов: Тестовый набор проверяет основные функции модуля конвертации: температурные шкалы, линейные преобразования, обработку ошибок и точность обратных вычислений.

*Листинг 6 — Код тестов.*

```

import pytest
import core.convert_script as conv

class TestConvertScript:

    def test_c_to_f_basic(self):
        assert conv.c_to_f(0) == 32.0
        assert conv.c_to_f(100) == 212.0

```

```

def test_f_to_c_basic(self):
    assert conv.f_to_c(32) == 0.0
    assert conv.f_to_c(212) == 100.0

def test_c_to_k_basic(self):
    assert conv.c_to_k(0) == 273.15
    assert conv.c_to_k(-273.15) == 0.0

def test_k_to_c_basic(self):
    assert conv.k_to_c(273.15) == 0.0
    assert conv.k_to_c(0) == -273.15

def test_convert_length_meter_to_km(self):
    result = conv.convert_linear("длина", 1000, "метр", "километр")
    assert result == 1.0

def test_convert_mass_gram_to_kg(self):
    result = conv.convert_linear("масса", 1000, "грамм", "килограмм")
    assert result == 1.0

def test_convert_unknown_category(self):
    with pytest.raises(ValueError):
        conv.convert_linear("объем", 100, "литр", "галлон")

def test_convert_unknown_unit(self):
    with pytest.raises(ValueError):
        conv.convert_linear("длина", 100, "фут", "метр")

def test_temperature_round_trip(self):
    original = 25.5
    converted = conv.f_to_c(conv.c_to_f(original))
    assert converted == pytest.approx(original, abs=1e-10)

def test_linear_round_trip(self):
    original = 150
    converted = conv.convert_linear("длина",
        conv.convert_linear("длина", original, "сантиметр", "метр"),
        "метр", "сантиметр")
    assert converted == pytest.approx(original, abs=1e-10)

if __name__ == "__main__":
    print(conv.c_to_f(0) == 32.0)

```

Методология: Тестирование реализовано на pytest с применением базовых проверок, обработки исключений и сравнительного анализа результатов.

Анализ покрытия кода: Тесты охватывают все ключевые функции конвертации, валидацию данных и обработку ошибок, включая позитивные и негативные сценарии.

Исправление ошибок:

Краткое описание ошибки: «Неверное преобразование цельсия в фаренгейты».

Статус ошибки: открыта («Open»).

Категория ошибки: серьезная («Major»).

Тестовый случай: «Проверка алгоритма функционирования программы».

Описание ошибки:

1. Загрузить программу.
  2. В поле ввода ввести строку «1».
  3. В поле ввода ввести строку «2».
  4. В поле ввода ввести строку «1».
  5. В поле ввода ввести строку «10».
4. Полученный результат: «122». Ожидаемый результат: «50».

*Листинг 7 — Исправленный код.*

```
# Единицы измерения
UNITS = {
    "длина": {
        "миллиметр": 10 ** -3,    # 0.001 метра
        "сантиметр": 10 ** -2,    # 0.01 метра
        "дециметр": 10 ** -1,     # 0.1 метра
        "метр": 1,                 # базовая единица
        "километр": 10 ** 3        # 1000 метров
    },
    "масса": {
        "грамм": 1,                # базовая единица
        "килограмм": 10 ** 3,      # 1000 грамм
        "центнер": 10 ** 5,       # 100 000 грамм
        "тонна": 10 ** 6           # 1 000 000 грамм
    }
}

# Функция преобразования цельсия в фаренгейты
def c_to_f(c):
    return c * 9.0 / 5.0 + 32.0

# Функция преобразования фаренгейтов в цельсии
def f_to_c(f):
    return (f - 32.0) * 5.0 / 9.0

# Функция преобразования цельсия в кельвины
def c_to_k(c):
    return c + 273.15
```

```

# Функция преобразования цельвинов в цельсии
def k_to_c(k):
    return k - 273.15

# Функция преобразования фааренгейтов в кельвины
def f_to_k(f):
    return (f - 32.0) * 5.0 / 9.0 + 273.15

# Функция преобразования кельвинов в фаренгейты
def k_to_f(k):
    return (k - 273.15) * 9.0 / 5.0 + 32.0

# Функция преобразования остальных категорий
def convert_linear(category, value, from_unit, to_unit):
    if category not in UNITS:
        raise ValueError("Неизвестная категория конверсии: " + str(category))

    units = UNITS[category]
    if from_unit not in units:
        raise ValueError("Неизвестная единица: " + from_unit)
    if to_unit not in units:
        raise ValueError("Неизвестная единица: " + to_unit)
    base_value = value * units[from_unit]
    result = base_value / units[to_unit]
    return result

```

## 6) Модуль «history»

Описание тестов: Тестовый набор проверяет функциональность модуля истории конвертаций, включая загрузку и сохранение данных, добавление записей, получение последних конвертаций, фильтрацию по категориям, очистку истории и формирование статистики. Особое внимание уделяется проверке корректности временных меток и сохранению данных в JSON-файл.

*Листинг 8 — Код тестов.*

```

import pytest
import json
import core.history as hm
from pathlib import Path

@pytest.fixture(autouse=True)
def clear_history_file(tmp_path, monkeypatch):
    test_file = tmp_path / "test_history.json"
    monkeypatch.setattr(hm, "FILENAME", test_file)

    if hasattr(hm, "_history_cache"):
        hm._history_cache = []

    if hasattr(hm, "clear_history"):
        try:
            hm.clear_history()
        except Exception:
            pass

```



```

yield

if test_file.exists():
    test_file.unlink()

def test_load_history_empty():
    assert hm.load_history() == []

def test_save_and_load_history():
    test_data = [{"test": "data"}]
    hm.save_history(test_data)
    loaded_data = hm.load_history()
    assert loaded_data == test_data

def test_add_conversion():
    hm.add_conversion("длина", 100, "см", "м", 1.0)
    history = hm.load_history()
    assert len(history) == 1
    assert history[0]["category"] == "длина"
    assert history[0]["value"] == 100
    assert history[0]["from_unit"] == "см"
    assert history[0]["to_unit"] == "м"
    assert history[0]["result"] == 1.0

def test_get_recent_conversions():
    for i in range(3):
        hm.add_conversion("длина", i, "м", "см", i * 100)
    recent = hm.get_recent_conversions(2)
    assert len(recent) == 2
    assert recent[0]["value"] == 1
    assert recent[1]["value"] == 2

def test_clear_history():
    hm.add_conversion("масса", 1000, "г", "кг", 1.0)
    hm.clear_history()
    assert hm.load_history() == []

def test_get_conversions_by_category():
    hm.add_conversion("длина", 100, "см", "м", 1.0)
    hm.add_conversion("масса", 1000, "г", "кг", 1.0)
    hm.add_conversion("длина", 2, "м", "км", 0.002)
    length_conv = hm.get_conversions_by_category("длина")
    assert len(length_conv) == 2
    for conv in length_conv:
        assert conv["category"] == "длина"

def test_get_statistics_empty():
    stats = hm.get_statistics()
    assert stats == {"total": 0}

def test_get_statistics_with_data():
    hm.add_conversion("длина", 100, "см", "м", 1.0)
    hm.add_conversion("масса", 1000, "г", "кг", 1.0)
    hm.add_conversion("длина", 2, "м", "км", 0.002)
    stats = hm.get_statistics()
    assert stats["total"] == 3
    assert stats["by_category"]["длина"] == 2
    assert stats["by_category"]["масса"] == 1

def test_timestamp_format():

```

```

hm.add_conversion("длина", 100, "см", "м", 1.0)
history = hm.load_history()
timestamp = history[0]["timestamp"]
assert "T" in timestamp
assert ":" in timestamp

def test_file_persistence():
    hm.add_conversion("длина", 150, "см", "м", 1.5)
    assert hm.FILENAME.exists()
    with open(hm.FILENAME, 'r', encoding='utf-8') as f:
        data = json.load(f)
    assert len(data) == 1
    assert data[0]["value"] == 150

```

Методология: Тестирование реализовано с использованием pytest и включает автоматическую очистку тестового файла истории перед каждым тестом. Проверяется как базовая функциональность (загрузка/сохранение), так и сложные сценарии работы с историей конвертаций, включая проверку структуры записей и их корректности.

Анализ покрытия кода: Тесты охватывают все ключевые функции модуля истории: работу с файлами, манипуляции с записями, формирование статистики и временные метки. Проверяется как позитивный сценарий (корректная работа с данными), так и граничные случаи, включая пустую историю и различные категории конвертаций.

Исправление ошибок:

Краткое описание ошибки: «Неверное считывание недавней истории конверсии.».

Статус ошибки: открыта («Open»).

Категория ошибки: критическая («Critical»).

Тестовый случай: «Проверка алгоритма функционирования программы».

Описание ошибки:

1. Запустить тест:

for i in range(5):

core.history.add\_conversion("длина", i, "м", "см", i \* 100)

```
recent = core.history.get_recent_conversions(2)
```

2. Полученный результат: «recent[0], recent[1], recent[2]». Ожидаемый результат: «recent[3], recent[4]».

*Листинг 9 — Исправленный код.*

```
import json
import datetime
from pathlib import Path
debug = False

FILENAME = Path("conversion_history.json")

def load_history():
    #Загрузить историю из файла
    if FILENAME.exists():
        with open(FILENAME, 'r', encoding='utf-8') as f:
            return json.load(f)
    return []

def save_history(history):
    #Сохранить историю в файл
    with open(FILENAME, 'w', encoding='utf-8') as f:
        json.dump(history, f, ensure_ascii=False, indent=2)
        print("Json has been saved.")
    if debug:
        print("-----")
        print(f"history: {history}")
        print("-----")

def add_conversion(category, value, from_unit, to_unit, result):
    #Добавить запись о конвертации
    history = load_history()
    entry = {
        "timestamp": datetime.datetime.now().isoformat(),
        "category": category,
        "value": value,
        "from_unit": from_unit,
        "to_unit": to_unit,
        "result": result
    }
    history.append(entry)
    save_history(history)

def get_recent_conversions(count=5):
    #Получить последние сколько-то конвертаций
    history = load_history()
    return history[-count:]

def clear_history():
    #Очистить всю историю
    save_history([])

def get_conversions_by_category(category):
    #Получить все конвертации по категории
    history = load_history()
    return [entry for entry in history if entry["category"] == category]
```

```

def get_statistics():
    #Получить статистику по истории
    history = load_history()
    if not history:
        return {"total": 0}

    categories = {}
    for entry in history:
        cat = entry["category"]
        categories[cat] = categories.get(cat, 0) + 1

    return {
        "total": len(history),
        "by_category": categories,
        "first_conversion": history[0]["timestamp"],
        "last_conversion": history[-1]["timestamp"]
    }

```

### в) Модуль «unit\_formatter»

Описание тестов: Тестовый набор проверяет функциональность модуля форматирования единиц измерения и результатов конвертации, включая форматирование чисел, склонение единиц измерения, обработку температурных шкал и формирование истории конвертаций.

*Листинг 10 — Код тестов.*

```

import pytest
from datetime import datetime
import core.unit_formatter as uf

def test_format_number_basic():
    assert uf.format_number(0) == "0"
    assert uf.format_number(3.14) == "3.14"
    assert uf.format_number(5.0) == "5"

def test_format_number_scientific():
    result = uf.format_number(1234567)
    assert "e" in result or result.isdigit()

def test_format_unit_name_plural():
    assert uf.format_unit_name("метр", 1) == "метр"
    assert uf.format_unit_name("метр", 2) == "метры"
    assert uf.format_unit_name("грамм", 5) == "граммы"

def test_format_temperature_symbols():
    from_sym, to_sym = uf.format_temperature(0, 'c', 'f')
    assert from_sym == '°C'
    assert to_sym == '°F'

def test_format_conversion_result_length():
    result = uf.format_conversion_result(100, "сантиметр", 1, "метр")
    assert "100 сантиметры = 1 метр" in result

def test_format_conversion_result_temperature():
    result = uf.format_conversion_result(0, "c", 32, "f")
    assert "0°C = 32°F" in result

```

```

def test_format_history_entry():
    entry = {
        'timestamp': '2024-01-15T14:30:00',
        'value': 100.0,
        'from_unit': 'сантиметр',
        'to_unit': 'метр',
        'result': 1.0
    }
    result = uf.format_history_entry(entry)
    assert "[15.01.2024 14:30]" in result
    assert "100 сантиметры = 1 метр" in result

def test_format_number_precision():
    assert uf.format_number(3.141592, precision=3) == "3.142"
    assert uf.format_number(3.141592, precision=5) == "3.14159"

def test_format_unit_name_unknown():
    assert uf.format_unit_name("литр", 2) == "литр"

def test_format_temperature_unknown():
    from_sym, to_sym = uf.format_temperature(100, 'rankine', 'unknown')
    assert from_sym == 'rankine'
    assert to_sym == 'unknown'

```

Методология: Тестирование построено на проверке базовых сценариев форматирования с использованием прямых сравнений результатов. Особое внимание уделяется корректности отображения чисел, единиц измерения и температурных символов, а также работе с различными форматами вывода.

Анализ покрытия кода: Тесты охватывают все ключевые функции форматирования: числовое представление, склонение единиц, обработку температурных шкал, формирование результатов конвертации и записи истории. Проверяется как стандартная, так и граничная функциональность модуля.

Исправление ошибок:

Краткое описание ошибки: «Неверное использование точности в функции форматирования в экспоненциальную запись».

Статус ошибки: открыта («Open»).

Категория ошибки: незначительная («Minor»).

Тестовый случай: «Проверка алгоритма функционирования программы».

Описание ошибки:

1. Загрузить тест.

```
result = core.unit_formatter.format_number(1234567)
```

2. Полученный результат: «1.234567e+6». Ожидаемый результат: «1.23456e+6».

*Листинг 11 — Исправленный код.*

```
import datetime

# Функция форматирования числа с заданной точностью
def format_number(value, precision=6):
    if value == 0:
        return "0"

    if abs(value) >= 1e6 or (abs(value) <= 1e-4 and value != 0):
        return f"{value:.{precision-1}e}"

    formatted = f"{value:.{precision}f}"
    if '.' in formatted:
        formatted = formatted.rstrip('0').rstrip('.')

    return formatted

# Функция форматирования названия единиц измерения
def format_unit_name(unit, value):
    if value == 1:
        return unit

    plural_rules = {
        'метр': 'метры',
        'километр': 'километры',
        'сантиметр': 'сантиметры',
        'грамм': 'граммы',
        'килограмм': 'килограммы',
        'тонна': 'тонны'
    }

    return plural_rules.get(unit, unit)

# Функция специального форматирования для температур
def format_temperature(value, from_unit, to_unit):
    symbols = {
        'c': '°C',
        'f': '°F',
        'k': 'K',
        'цельсий': '°C',
        'фаренгейт': '°F',
        'кельвин': 'K'
    }

    from_symbol = symbols.get(from_unit.lower(), from_unit)
    to_symbol = symbols.get(to_unit.lower(), to_unit)

    return from_symbol, to_symbol
```

```

# Функция форматирования полного результата конвертации
def format_conversion_result(value, from_unit, result, to_unit):
    formatted_value = format_number(value)
    formatted_result = format_number(result)

    if any(unit in ['c', 'f', 'k', 'цельсий', 'фаренгейт', 'кельвин']
           for unit in [from_unit, to_unit]):
        from_sym, to_sym = format_temperature(value, from_unit, to_unit)
        return f"{formatted_value}{from_sym} = {formatted_result}{to_sym}"

    from_name = format_unit_name(from_unit, value)
    to_name = format_unit_name(to_unit, result)

    return f"{formatted_value} {from_name} = {formatted_result} {to_name}"

# Функция форматирования записи истории для красивого вывода
def format_history_entry(entry):
    timestamp = datetime.datetime.fromisoformat(entry['timestamp'])
    date_str = timestamp.strftime("%d.%m.%Y %H:%M")
    result_str = format_conversion_result(
        entry['value'],
        entry['from_unit'],
        entry['result'],
        entry['to_unit']
    )
    return f"[{date_str}] {result_str}"

```

### г) Модуль «validator»

Описание тестов: Тестовый набор направлен на проверку корректности работы модуля валидации входных данных, включая проверку числовых значений, их положительности, температурных диапазонов, существования единиц измерения и категорий конвертации.

*Листинг 12 — Код тестов.*

```

import pytest
import sys
import os
import core.validator as val

def test_validate_numeric_input_valid():
    assert val.validate_numeric_input("42") == 42.0
    assert val.validate_numeric_input("3.14") == 3.14
    assert val.validate_numeric_input("-10") == -10.0

def test_validate_numeric_input_invalid():
    with pytest.raises(ValueError, match="Ошибка: введите корректное число"):
        val.validate_numeric_input("abc")

def test_validate_positive_number_valid():
    assert val.validate_positive_number(5) == 5
    assert val.validate_positive_number(0) == 0
    assert val.validate_positive_number(0.001) == 0.001

def test_validate_positive_number_invalid():

```

```

    with pytest.raises(ValueError, match="Ошибка: значение должно быть
положительным"):
        val.validate_positive_number(-5)

def test_validate_temperature_range_valid():
    assert val.validate_temperature_range(0, 'c') == 0
    assert val.validate_temperature_range(-273.15, 'c') == -273.15
    assert val.validate_temperature_range(32, 'f') == 32
    assert val.validate_temperature_range(273.15, 'k') == 273.15

def test_validate_temperature_range_invalid():
    with pytest.raises(ValueError):
        val.validate_temperature_range(-274, 'c')
    with pytest.raises(ValueError):
        val.validate_temperature_range(-1, 'k')

def test_validate_unit_exists_valid():
    assert val.validate_unit_exists("длина", "метр") == True
    assert val.validate_unit_exists("масса", "грамм") == True

def test_validate_unit_exists_invalid():
    with pytest.raises(ValueError):
        val.validate_unit_exists("длина", "фут")
    with pytest.raises(ValueError):
        val.validate_unit_exists("объем", "литр")

def test_validate_category_valid():
    assert val.validate_category("длина") == True
    assert val.validate_category("масса") == True

def test_validate_category_invalid():
    with pytest.raises(ValueError):
        val.validate_category("объем")

```

Методология: Тестирование осуществляется через комбинацию проверок корректных и некорректных входных данных с использованием механизма перехвата исключений. Особое внимание уделяется валидации граничных значений и обработке ошибок при работе с различными единицами измерений.

Анализ покрытия кода: Тесты охватывают все основные функции валидации: проверку числовых значений, контроль положительности чисел, валидацию температурных диапазонов для разных шкал, проверку существования единиц измерения и категорий конвертации. Проверяется как корректная работа с допустимыми значениями, так и обработка ошибочных ситуаций.

Исправление ошибок:

Краткое описание ошибки: «Неверная проверка положительного числа».



Статус ошибки: открыта («Open»).

Категория ошибки: серьезная («Major»).

Тестовый случай: «Проверка алгоритма функционирования программы».

Описание ошибки:

1. Загрузить тест.

```
result = core.validator.validate_positive_number(0)
```

2. Полученный результат: «ValueError("Ошибка: значение должно быть положительным")». Ожидаемый результат: «0».

*Листинг 13 — Исправленный код.*

```
# Функция проверки является ли числом
def validate_numeric_input(value_str):
    try:
        return float(value_str)
    except ValueError:
        raise ValueError("Ошибка: введите корректное число")

# Функция проверки что число положительное
def validate_positive_number(value):
    if value < 0:
        raise ValueError("Ошибка: значение должно быть положительным")
    return value

# Функция проверки допустимости температуры
def validate_temperature_range(value, scale):
    ranges = {
        'c': (-273.15, 10000),
        'f': (-459.67, 18032),
        'k': (0, 10273.15)
    }
    min_temp, max_temp = ranges.get(scale, (-float('inf'), float('inf')))
    if not min_temp <= value <= max_temp:
        raise ValueError(f"Температура должна быть в диапазоне {min_temp} - {max_temp}")
    return value

# Функция проверки существования единицы измерения
def validate_unit_exists(category, unit):
    from core.convert_script import UNITS
    if category not in UNITS:
        raise ValueError(f"Категория '{category}' не найдена")
    if unit not in UNITS[category]:
        raise ValueError(f"Единица '{unit}' не найдена в категории '{category}'")
    return True

# Функция проверки существования категории
def validate_category(category):
    from core.convert_script import UNITS
    if category not in UNITS:
        raise ValueError(f"Категория '{category}' не найдена")
```

```
return True
```

#### д) Модуль «view»

Описание тестов: Тестовый набор охватывает функциональность пользовательского интерфейса приложения, включая навигацию по меню, конвертацию линейных и температурных величин, работу с историей конвертаций и обработку ошибок валидации входных данных.

*Листинг 14 — Код тестов.*

```
import pytest
import sys
import os
import core.view as view
from unittest.mock import patch, MagicMock

@patch('builtins.input')
@patch('builtins.print')
def test_menu_navigation(mock_print, mock_input):
    mock_input.return_value = '1'
    with patch('core.view.convert_menu') as mock_convert:
        view.menu()
        mock_convert.assert_called_once()

@patch('builtins.input')
@patch('builtins.print')
def test_menu_invalid_input(mock_print, mock_input):
    mock_input.return_value = 'abc'
    view.menu()
    mock_print.assert_any_call("Введите число!")

@patch('builtins.input')
@patch('builtins.print')
def test_convert_menu_linear(mock_print, mock_input):
    mock_input.return_value = '1'
    with patch('core.view.convert_linear_menu') as mock_linear:
        view.convert_menu()
        mock_linear.assert_called_with('длина')

@patch('builtins.input')
@patch('builtins.print')
def test_convert_linear_menu_success(mock_print, mock_input):
    mock_input.side_effect = ['100', 'сантиметр', 'метр']

    with patch('core.validator.validate_numeric_input') as mock_val_num, \
        patch('core.validator.validate_positive_number') as mock_val_pos, \
        patch('core.validator.validate_unit_exists') as mock_val_unit, \
        patch('core.conversions.convert_linear') as mock_convert, \
        patch('core.unit_formatter.format_conversion_result') as mock_format, \
        patch('core.history.add_conversion') as mock_history:

        mock_val_num.return_value = 100.0
        mock_val_pos.return_value = 100.0
        mock_convert.return_value = 1.0
        mock_format.return_value = "100 см = 1 м"
```

```

        view.convert_linear_menu('длина')

        mock_convert.assert_called_once()
        mock_history.assert_called_once()
        mock_print.assert_any_call("\n100 см = 1 м\n")

@patch('builtins.input')
@patch('builtins.print')
def test_convert_temperature_menu_success(mock_print, mock_input):
    mock_input.side_effect = ['1', '0']

    with patch('core.validator.validate_numeric_input') as mock_val_num, \
        patch('core.conversions.c_to_f') as mock_convert, \
        patch('core.unit_formatter.format_conversion_result') as mock_format, \
        patch('core.history.add_conversion') as mock_history:

        mock_val_num.side_effect = [1.0, 0.0]
        mock_convert.return_value = 32.0
        mock_format.return_value = "0°C = 32°F"

        view.convert_temperature_menu()

        mock_convert.assert_called_once_with(0.0)
        mock_history.assert_called_once()

@patch('core.history.get_recent_conversions')
@patch('builtins.print')
def test_show_recent_history(mock_print, mock_get_recent):
    mock_get_recent.return_value = [{
        'timestamp': '2024-01-15T10:00:00',
        'category': 'длина',
        'value': 100.0,
        'from_unit': 'см',
        'to_unit': 'м',
        'result': 1.0
    }]

    view.show_recent_history()
    mock_print.assert_any_call("\nПоследние 5 конвертаций:")

@patch('core.history.get_statistics')
@patch('builtins.print')
def test_show_statistics(mock_print, mock_get_stats):
    mock_get_stats.return_value = {
        'total': 5,
        'by_category': {'длина': 3, 'масса': 2},
        'first_conversion': '2024-01-15T10:00:00',
        'last_conversion': '2024-01-15T11:00:00'
    }

    view.show_statistics()
    mock_print.assert_any_call("Всего конвертаций: 5")

@patch('builtins.input')
@patch('builtins.print')
def test_history_menu_clear(mock_print, mock_input):
    mock_input.side_effect = ['4', '0']

    with patch('core.history.clear_history') as mock_clear:
        view.history_menu()

```

```

        mock_clear.assert_called_once()
        mock_print.assert_any_call("История очищена.")

@patch('builtins.input')
@patch('builtins.print')
def test_convert_linear_menu_validation_error(mock_print, mock_input):
    mock_input.side_effect = ['invalid', 'см', 'м']

    with patch('core.validator.validate_numeric_input') as mock_val_num:
        mock_val_num.side_effect = ValueError("Ошибка числа")

        view.convert_linear_menu('длина')
        mock_print.assert_any_call("Ошибка:", "Ошибка числа")

@patch('builtins.input')
@patch('builtins.print')
def test_convert_temperature_invalid_choice(mock_print, mock_input):
    mock_input.side_effect = ['7', '0']

    with patch('core.validator.validate_numeric_input') as mock_val_num:
        mock_val_num.return_value = 7.0

        view.convert_temperature_menu()
        mock_print.assert_any_call("Неверный выбор!")

```

Методология: Тестирование реализовано с использованием моков и заглушек для имитации пользовательского ввода и вывода, а также для изоляции тестируемого кода от внешних зависимостей. Проверяется как корректная работа интерфейса, так и обработка ошибочных ситуаций.

Анализ покрытия кода: Тесты охватывают все основные компоненты пользовательского интерфейса: главное меню, меню конвертации, обработку различных типов конвертаций, работу с историей и статистикой. Проверяется корректность отображения результатов, обработка ошибок валидации и взаимодействие с другими модулями приложения. Особое внимание уделяется тестированию граничных случаев и обработке некорректного ввода.

Исправление ошибок:

Краткое описание ошибки: «Неверное преобразование Цельсия в Фаренгейты».

Статус ошибки: открыта («Open»).

Категория ошибки: серьезная («Major»).

Тестовый случай: «Проверка алгоритма функционирования программы».

Описание ошибки:

1. Загрузить программу.
2. В поле ввода ввести строку «1».
2. В поле ввода ввести строку «2».
2. В поле ввода ввести строку «1».
2. В поле ввода ввести строку «10».
2. В поле ввода ввести строку «0».
2. В поле ввода ввести строку «0».
2. В поле ввода ввести строку «2».
2. В поле ввода ввести строку «1».
4. Полученный результат: «-12.22». Ожидаемый результат: «50».

Листинг 15 — Исправленный код.

```
import time
import core.convert_script as conversions
import core.validator as validator
import core.unit_formatter as unit_formatter
import core.history as history

def startup():
    print(r"""
/$$$$$\  $$$$$$\  $$\  $$\  $$\  $$\  $$$$$$$$\  $$$$$$$$\  $$$$$$$$\  $$$$$$$$\  $$$
$$$$$\ 
$$  __$$\  $$  __$$\  $$$\  $$ |$$ |  $$ |$$  ____|$$  __$$\  __$$  __|$$  ____|$$
__$$\ 
$$ /  \__|$$ /  $$ |$$$$\  $$ |$$ |  $$ |$$  ____|  $$ |  $$ |  $$ |  $$ |  $$
|  $$ |
$$ |  $$ |  $$ |$$ $$$\  $\$  \  $$ |$$$$$\  $$$$$$$  |  $$ |  $$$$$\  $$
$$$$$  |
$$ |  $$ |  $$ |$$ \$$$$ | \$$\$$ /  $$  __|  $$  __$$<  $$ |  $$  __|  $$
__$$<
$$ |  $$\  $$ |  $$ |$$ |\$$$$ | \$$$ /  $$ |  $$ |  $$ |  $$ |  $$
|  $$ |
\$$$$$  | $$$$$$  |$$ | \$$ |  \$/  $$$$$$$\  $$ |  $$ |  $$ |  $$$$$$$\  $$
|  $$ |
 \_____/  \_____/  \_  \_  \_  \_____| \_  \_  \_  \_____| \
__|  \__|
""")
    time.sleep(1)
    while True:
        menu()
```

```

#Главное меню
def menu():
    print()
    print("Что вы хотите сделать?")
    print("1. Конвертировать")
    print("2. Посмотреть историю")
    print("0. Выйти")

    try:
        choice = int(input(">> "))
        if choice == 1:
            convert_menu()
        elif choice == 2:
            history_menu()
        elif choice == 0:
            exit()
        else:
            print("Неверный ввод!")
    except ValueError:
        print("Введите число!")

#Меню истории
def history_menu():
    while True:
        print("\nИстория конвертаций:")
        print("1. Последние 5 конвертаций")
        print("2. Вся история")
        print("3. Статистика")
        print("4. Очистить историю")
        print("0. Назад")
        choice = input(">> ")
        if choice == "1":
            show_recent_history()
        elif choice == "2":
            show_full_history()
        elif choice == "3":
            show_statistics()
        elif choice == "4":
            history.clear_history()
            print("История очищена.")
        elif choice == "0":
            return
        else:
            print("Неверный ввод!")

def show_recent_history():
    data = history.get_recent_conversions(5)
    if not data:
        print("История пуста.")
        return
    print("\nПоследние 5 конвертаций:")
    for entry in data:
        print(
            f"[{entry['timestamp']}] {entry['category']}: {entry['value']} "
            f"{entry['from_unit']} → {entry['result']} {entry['to_unit']}"
        )

def show_full_history():
    data = history.load_history()
    if not data:
        print("История пуста.")

```

```

        return
    print("\nВся история:")
    for entry in data:
        print(
            f"[{entry['timestamp']}] {entry['category']}: {entry['value']} "
            f"{entry['from_unit']} → {entry['result']} {entry['to_unit']}")

def show_statistics():
    stats = history.get_statistics()
    if stats["total"] == 0:
        print("История пуста.")
        return
    print("\nСтатистика:")
    print(f"Всего конвертаций: {stats['total']}")
    print("По категориям:")
    for cat, count in stats["by_category"].items():
        print(f" - {cat}: {count}")
    print(f"Первая: {stats['first_conversion']}")
    print(f"Последняя: {stats['last_conversion']}")

#Меню выбора типа конвертации
def convert_menu():
    print()
    print("Что вы хотите конвертировать?")
    print("1. Длина")
    print("2. Температура")
    print("3. Масса")
    print("0. Назад")

    choice = input(">> ")
    if choice == "1":
        convert_linear_menu("длина")
    elif choice == "2":
        convert_temperature_menu()
    elif choice == "3":
        convert_linear_menu("масса")
    elif choice == "0":
        return
    else:
        print("Неверный ввод!")

#Конвертации длины/массы
def convert_linear_menu(category):
    print("\nДоступные единицы:")
    for unit in conversions.UNITS[category]:
        print(" -", unit)

    try:
        value_str = input("\nВведите значение: ")
        value = validator.validate_numeric_input(value_str)
        value = validator.validate_positive_number(value)

        from_unit = input("Из (например, метр): ").strip().lower()
        to_unit = input("В (например, километр): ").strip().lower()

        validator.validate_unit_exists(category, from_unit)
        validator.validate_unit_exists(category, to_unit)

        result = conversions.convert_linear(category, value, from_unit, to_unit)
        formatted_result = unit_formatter.format_conversion_result(value, from_unit,

```

```

result, to_unit)
    print(f"\n{formatted_result}\n")

    history.add_conversion(category, value, from_unit, to_unit, result)

except Exception as e:
    print("Ошибка:", e)

#Конвертации температур
def convert_temperature_menu():
    print("""
1. Цельсий --> Фаренгейт
2. Фаренгейт --> Цельсий
3. Цельсий --> Кельвин
4. Кельвин --> Цельсий
5. Фаренгейт --> Кельвин
6. Кельвин --> Фаренгейт
0. Назад
""")
    try:
        str_choice = input(">> ")
        choice = validator.validate_numeric_input(str_choice)

        if choice == 0:
            return

        str_value = input("Введите значение: ")
        value = validator.validate_numeric_input(str_value)

        conversions_map = {
            1: ("цельсий", "фаренгейт", conversions.c_to_f),
            2: ("фаренгейт", "цельсий", conversions.f_to_c),
            3: ("цельсий", "кельвин", conversions.c_to_k),
            4: ("кельвин", "цельсий", conversions.k_to_c),
            5: ("фаренгейт", "кельвин", conversions.f_to_k),
            6: ("кельвин", "фаренгейт", conversions.k_to_f)
        }

        if choice not in conversions_map:
            print("Неверный выбор!")
            return

        from_unit, to_unit, func = conversions_map[choice]
        value = validator.validate_temperature_range(value, from_unit[0])
        result = func(value)

        formatted_result = unit_formatter.format_conversion_result(value, from_unit,
result, to_unit)
        print(f"\n{formatted_result}\n")

        history.add_conversion("температура", value, from_unit, to_unit, result)

    except ValueError:
        print("Ошибка: введите число!")

if __name__ == "__main__":
    startup()

```



## 2.3 Мутационное тестирование

### а) Модуль «convert»

Создание мутантов:

1. Мутации в функциях конвертации температур:

| Функция   | Исходный код                      | Мутант (Ошибка)                                      | Результат теста                                     | Убит ?                                                                                       | Недостаток теста? |
|-----------|-----------------------------------|------------------------------------------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------|-------------------|
| c_to_f(c) | $c * 9.0 / 5.0 + 32.0$            | $c * 9.0 / 5.0 + 32.1$<br>(Изменение константы)      | test_c_to_f_basics<br>Провалится (32.1 != 32.0)     | <br>Да    | Нет               |
| f_to_c(f) | $(f - 32.0) * 5.0 / 9.0$          | $(f - 32.0) * 5.0 / 9.1$<br>(Изменение константы)    | test_f_to_c_basics<br>Провалится                    | <br>Да    | Нет               |
| c_to_k(c) | $c + 273.15$                      | $c + 273.16$<br>(Изменение константы)                | test_c_to_k_basics<br>Провалится (273.16 != 273.15) | <br>Да  | Нет               |
| k_to_c(k) | $k - 273.15$                      | $k + 273.15$<br>(Замена оператора)                   | test_k_to_c_basics<br>Провалится                    | <br>Да  | Нет               |
| k_to_f(k) | $(k - 273.15) * 9.0 / 5.0 + 32.0$ | $(k - 273.15) * 9.0 / 5.0 - 32.0$<br>(Замена + на -) | Выживет при k=273.15!                               | <br>Нет | Да                |
| f_to_k(f) | $(f - 32.0) * 5.0 / 9.0 + 273.15$ | $(f - 32.0) * 5.0 / 9.1 + 273.15$                    | Выживет при k=273.15!                               | <br>Нет | Да                |

| Функция | Исходный код             | Мутант (Ошибка)                          | Результат теста | Убит ? | Недостаток теста? |
|---------|--------------------------|------------------------------------------|-----------------|--------|-------------------|
|         | 5.0 /<br>9.0 +<br>273.15 | 5.0 / 9.0<br>- 273.15<br>(Замена + на -) | f=32.0!         |        |                   |

## 2. Мутации в Функции convert\_linear:

| Код | Исходный код                                | Мутант (Ошибка)                                                   | Результат теста                                                                               | Убит ?                                                                                      | Недостаток теста? |
|-----|---------------------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|-------------------|
| A   | if category<br>not in UNITS:                | if<br>category<br>in<br>UNITS:<br>(Замена<br>not in<br>на in)     | test_convert_unknown_category<br>Провалится<br>(ожидали<br>ValueError,<br>но его не<br>будет) | <br>Да   | Нет               |
| B   | raise<br>ValueError(...<br>)                | Удалить<br>строку<br>(Удаление<br>оператора<br>)                  | test_convert_unknown_category<br>Провалится<br>(тест ожидает<br>исключение,<br>но его нет)    | <br>Да | Нет               |
| C   | base_value =<br>value *<br>units[from_unit] | base_value =<br>value +<br>units[from_unit]<br>(Замена *<br>на +) | test_convert_length_meter_to_k<br>m Провалится<br>(1000*1 !=<br>1000+1)                       | <br>Да | Нет               |
| D   | result =<br>base_value /<br>units[to_unit]  | result =<br>base_value *                                          | test_convert_length_meter_to_k<br>m Провалится                                                | <br>Да | Нет               |

| К<br>о<br>д | Исходный код                  | Мутант<br>(Ошибка)                                              | Результат<br>теста                                                                                | Убит<br>?                                                                                 | Недоста<br>ток<br>теста? |
|-------------|-------------------------------|-----------------------------------------------------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|--------------------------|
|             |                               | units[t<br>o_unit]<br>(Замена /<br>на *)                        | (1000/1000 !=<br>1000*1000)                                                                       |                                                                                           |                          |
| E           | if from_unit<br>not in units: | if<br>from_un<br>it in<br>units:<br>(Замена<br>not in<br>на in) | test_conve<br>rt_unknown<br>_unit<br>Провалится<br>(ожидали<br>ValueError,<br>но его не<br>будет) | <br>Да | Нет                      |

Анализ их выживания:

Тесты test\_c\_to\_f\_basic и test\_f\_to\_c\_basic хорошо покрывают крайние точки (0°C/32°F и 100°C/212°F). Однако, для функций k\_to\_f и f\_to\_k, мутанты, меняющие финальный знак (+ 32.0 на - 32.0 и + 273.15 на - 273.15), выжили при входном значении, которое обнуляет первую часть формулы (32°F или 273.15K).

Корректировка тестов:

1. Тест для k\_to\_f: Проверить известную точку, где первая часть формулы не равна нулю.

*Листинг 16 — Код теста.*

```
def test_k_to_f_basic(self):
    # 0K = -459.67F
    assert conv.k_to_f(0) == pytest.approx(-459.67, abs=1e-10)
    # 288.15K = 59.0F
    assert conv.k_to_f(288.15) == 59.0
```

2. Тест для f\_to\_k: Проверить известную точку, где разность не равна нулю.

*Листинг 17 — Код теста.*

```
def test_f_to_k_basic(self):
    # 32F = 273.15K
```

```
assert conv.f_to_k(32) == 273.15
# 59F = 288.15K
assert conv.f_to_k(59) == 288.15
```

## 6) Модуль «history»

Создание мутантов:

### 1. Мутации в load\_history

| Исходный код             | Мутант (Ошибка)                                               | Результат теста                                                                                 | Убит?                                                                                       | Недостаток теста? |
|--------------------------|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|-------------------|
| if<br>FILENAME.exists(): | if not<br>FILENAME.exists(): (Инверсия условия)               | test_load_history_empty<br>Провалится (файл не существует, но функция попытается его прочитать) | <br>Да   | Нет               |
| return []                | return [1]<br>(Изменение возвращаемого значения по умолчанию) | test_load_history_empty<br>Провалится (ожидается []), возвращается [1])                         | <br>Да | Нет               |

### 2. Мутации в save\_history

| Исходный код                         | Мутант (Ошибка)                       | Результат теста            | Убит?                                                                                        | Недостаток теста? |
|--------------------------------------|---------------------------------------|----------------------------|----------------------------------------------------------------------------------------------|-------------------|
| print("Json has been saved.")        | Удаление строки (Удаление оператора)  | Выживет                    | <br>Нет | Да                |
| with open(FILENAME, 'w', encoding='u | with open(FILENAME, 'a', encoding='ut | test_save_and_load_history | <br>Нет | Да                |

| Исходный код | Мутант (Ошибка)                  | Результат теста                                                               | Убит ? | Недостаток теста? |
|--------------|----------------------------------|-------------------------------------------------------------------------------|--------|-------------------|
| tf-8')       | f-8') (Замена режима 'w' на 'a') | Выживет (При первом запуске тестов файл пуст, 'w' и 'a' ведут себя одинаково) |        |                   |

### 3. Мутации в add\_conversion и get\_recent\_conversions

| Функция                | Исходный код                                             | Мутант (Ошибка)                                    | Результат теста                                                                    |
|------------------------|----------------------------------------------------------|----------------------------------------------------|------------------------------------------------------------------------------------|
| add_conversion         | history.append(entry)                                    | Удаление строки (Удаление оператора)               | test_add_conversion<br>Провалится (len(history) будет 0)                           |
| add_conversion         | "timestamp":<br>datetime.datetime.now().<br>isoformat(), | "timestamp":<br>"123",<br>(Изменение логики поля)  | test_timestamp_format<br>Провалится ("T" или ":" не будет в "123")                 |
| get_recent_conversions | return history[-count:]                                  | return<br>history[:<br>count]<br>(Изменение среза) | test_get_recent_conversions<br>Провалится (вернет первые 2 записи, а не последние) |
| get_recent_conversions | return history[-count:]                                  | return<br>history                                  | test_get_recent_conversions                                                        |

| Функция | Исходный код | Мутант (Ошибка)  | Результат теста                                             |
|---------|--------------|------------------|-------------------------------------------------------------|
| ions    |              | (Удаление среза) | nversions<br>Провалится<br>(вернет 3<br>записи<br>вместо 2) |

#### 4. Мутации в get\_conversions\_by\_category

| Исходный код                  | Мутант (Ошибка)                                       | Результат теста                                                                           | Убит?                                                                                       | Недостаток теста? |
|-------------------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|-------------------|
| entry["category"] == category | entry["category"] != category<br>(Инверсия сравнения) | test_get_conversions_by_category<br>Провалится<br>(длина будет 1 (масса), а не 2 (длина)) | <br>Да   | Нет               |
| entry["category"] == category | entry["value"] == category<br>(Изменение ключа)       | test_get_conversions_by_category<br>Провалится<br>(длина будет 0, т.к. 100 != "длина")    | <br>Да | Нет               |

#### 5. Мутации в get\_statistics

| Исходный код    | Мутант (Ошибка)                   | Результат теста                                                                 | Убит?                                                                                       | Недостаток теста? |
|-----------------|-----------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|-------------------|
| if not history: | if history:<br>(Инверсия условия) | test_get_statistics_empty<br>Провалится<br>(будет ошибка IndexError при попытке | <br>Да | Нет               |

| Исходный код               | Мутант (Ошибка)                                      | Результат теста                                                        | Убит ?                                                                                     | Недостаток теста? |
|----------------------------|------------------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|-------------------|
|                            |                                                      | получить history[0])                                                   |                                                                                            |                   |
| categories.get(cat, 0) + 1 | categories.get(cat, 0) + 2<br>(Изменение приращения) | test_get_statistics_with_data<br>Провалится (счетчик будет 3 вместо 2) | <br>Да  | Нет               |
| history[0] ["timestamp"]   | history[-1] ["timestamp"]<br>(Изменение индекса)     | test_get_statistics_with_data<br>Выживет                               | <br>Нет | Да                |

Анализ их выживания:

Мутант 1 (Удаление print): Тесты не проверяют вывод в консоль, что является ожидаемым поведением, поскольку это не влияет на функциональную логику. Мутант выживает, но это эквивалентный мутант или мутант, не имеющий отношения к бизнес-логике.

Мутант 2 (Режим записи): Если заменить 'w' (перезапись) на 'a' (добавление), тесты, которые добавляют записи (например, test\_add\_conversion), будут работать неправильно, если запускались бы без очистки между собой. Однако test\_save\_and\_load\_history выживает, потому что файл пуст.

Мутант 3 (Статистика): Мутант, который менял history[0] на history[-1] при получении first\_conversion, выжил. Это случилось потому, что наш тест test\_get\_statistics\_with\_data не проверял фактические значения first\_conversion и last\_conversion, а фокусировался только на общих подсчетах.

Корректировка тестов:

1. Чтобы убить выжившего мутанта в `get_statistics`, мы добавили явную проверку того, что `first_conversion` и `last_conversion` действительно содержат разные значения.

*Листинг 18 — Код теста.*

```
def test_get_statistics_timestamps(self):
    # Добавляем две конвертации, чтобы убедиться, что они имеют разное время
    hm.add_conversion("длина", 100, "см", "м", 1.0)
    # Искусственная задержка для гарантированно разного timestamp
    import time; time.sleep(0.01)
    hm.add_conversion("масса", 1000, "г", "кг", 1.0)

    stats = hm.get_statistics()

    # 1. Проверяем, что первый и последний timestamp разные
    assert stats["first_conversion"] != stats["last_conversion"]

    # 2. Проверяем, что первый - это первый, а последний - это последний
    (опционально, но желательно)
    history = hm.load_history()
    assert stats["first_conversion"] == history[0]["timestamp"]
    assert stats["last_conversion"] == history[-1]["timestamp"]
```

2. Чтобы убить выжившего мутанта в `save_history` (Режим 'w' vs 'a'), мы убедились, что один тест сохраняет данные, а следующий загружает их и обнаруживает, что старые данные были удалены (т.е. что произошла перезапись).

*Листинг 19 — Код теста.*

```
def test_save_history_overwrites(self):
    # 1. Сохраняем первую запись
    hm.save_history([{"id": 1}])
    # 2. Сохраняем вторую запись (должна перезаписать)
    hm.save_history([{"id": 2}])

    loaded_data = hm.load_history()
    # Если бы был режим 'a', то len было бы 2. В режиме 'w' должно быть 1.
    assert len(loaded_data) == 1
    assert loaded_data[0]["id"] == 2
```

## в) Модуль «unit\_formater»

Создание мутантов:

### 1. Мутации в `format_number`

| Исходный код                | Мутант (Ошибка)             | Результат теста       | Убит ?                                                                                | Недостаток теста? |
|-----------------------------|-----------------------------|-----------------------|---------------------------------------------------------------------------------------|-------------------|
| <code>if value == 0:</code> | <code>if value != 0:</code> | <code>test_for</code> |  | Нет               |



| Исходный код                         | Мутант (Ошибка)                                             | Результат теста                                                                                | Убит ?                                                                                       | Недостаток теста? |
|--------------------------------------|-------------------------------------------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-------------------|
|                                      | (Инверсия условия)                                          | mat_number_basic<br>(для value=0)<br>Провалится<br>(не вернет "0")                             | Да                                                                                           |                   |
| abs(value) >= 1e6                    | abs(value) > 1e6 (Изменение оператора)                      | Выживет                                                                                        | <br>Нет   | Да                |
| formatted.rstrip('0').rstrip('.')    | formatted.rstrip('0') (Удаление rstrip('.'))                | test_format_number_basic<br>(для 5.0)<br>Выживет<br>(вернет "5."<br>вместо "5")                | <br>Нет   | Да                |
| formatted = f"{value:.{precision}f}" | formatted = f"{value:.{precision-1}f}" (Изменение точности) | test_format_number_precision (для precision=5)<br>Выживет<br>(вернет 3.1416<br>вместо 3.14159) | <br>Нет | Да                |

## 2. Мутации в format\_unit\_name

| Исходный код   | Мутант (Ошибка)          | Результат теста        | Убит ?                                                                                      | Недостаток теста? |
|----------------|--------------------------|------------------------|---------------------------------------------------------------------------------------------|-------------------|
| if value == 1: | if value != 1: (Инверсия | test_format_unit_name_ | <br>Да | Нет               |

| Исходный код                            | Мутант (Ошибка)                                                             | Результат теста                                                                           | Убит?                                                                                       | Недостаток теста? |
|-----------------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|-------------------|
|                                         | условия)                                                                    | plural (для value=1)<br>Провалится (вернет "метры" вместо "метр")                         |                                                                                             |                   |
| return unit<br>(Единственное число)     | return unit + 's'<br>(Добавление 's')                                       | test_format_unit_name_plural (для value=1)<br>Провалится (вернет "meters" вместо "meter") | <br>Да   | Нет               |
| return plural_rule<br>s.get(unit, unit) | return plural_rules.get(unit, 'error')<br>(Изменение значения по умолчанию) | test_format_unit_name_unknown<br>Провалится (вернет "error" вместо "литр")                | <br>Да | Нет               |

### 3. Мутации в format\_conversion\_result

| Исходный код                            | Мутант (Ошибка)                                             | Результат теста                                    | Убит?                                                                                        | Недостаток теста? |
|-----------------------------------------|-------------------------------------------------------------|----------------------------------------------------|----------------------------------------------------------------------------------------------|-------------------|
| if<br>any(unit in ['c', 'f', 'k', ...]) | if all(unit in ['c', 'f', 'k', ...])<br>(Замена any на all) | Выживет                                            | <br>Нет | Да                |
| from_name =<br>format_unit_name(f       | from_name =<br>from_unit<br>(Удаление форматирования)       | test_format_conversion_result_length<br>Провалится | <br>Да  | Нет               |

| Исходный код     | Мутант (Ошибка) | Результат теста                                             | Убит ? | Недостаток теста? |
|------------------|-----------------|-------------------------------------------------------------|--------|-------------------|
| rom_unit, value) | )               | (вернет "100 сантиметр = ...", а не "100 сантиметры = ...") |        |                   |

Анализ их выживания:

Мутант 1 ( $\text{abs}(\text{value}) \geq 1\text{e}6$ ): Наш тест не включал пограничное значение  $10^6$ . Если изменить оператор, при  $10^6$  функция не перейдет в научную нотацию, что стало бы ошибкой.

Мутант 2 (Удаление `rstrip('.')`): Мутант выживает, потому что наш тест `test_format_number_basic` для 5.0 проверяет, что результат равен "5", но если удалить `rstrip('.')`, результат будет "5.". Тест должен быть уточнен.

Мутант 3 (Точность): Если уменьшить `precision` на 1, тест `test_format_number_precision` для  $3.141592$  при `precision=5` должен был бы провалиться, но он проверяет только, что результат округлен (что может быть верным и для `precision=4`).

Мутант 4 (Замена `any` на `all`): Мутант выживет, если наш тест проверяет температуру, где обе единицы являются известными символами (например, `c` и `f`). Если бы мы передали неизвестную единицу (например, `c` и `rankine`), `all` провалился бы, но `any` сработал бы. Наш тест `test_format_conversion_result_temperature` использует `c` и `f`, что делает его нечувствительным к этой мутации.

Корректировка тестов:

1. Улучшение `test_format_number_basic` (Мутант 2)

Тест должен явно проверить, что форматирование убирает конечную точку.

*Листинг 20 — Код теста.*

```
def test_format_number_basic():
    assert uf.format_number(0) == "0"
    assert uf.format_number(3.14) == "3.14"
    assert uf.format_number(5.0) == "5" # Убивает мутанта с удалением rstrip('.')
    # Дополнительная проверка на число с более длинным хвостом
    assert uf.format_number(123.45000) == "123.45"
```

## 2. Добавление теста для научных обозначений (Мутант 1)

Проверить пограничные значения для научной нотации.

*Листинг 21 — Код теста.*

```
def test_format_number_scientific_boundary():
    # Проверка, что 1e6 переходит в научную (>= 1e6)
    assert 'e' in uf.format_number(1000000, precision=6)
    # Проверка на очень маленькое число (<= 1e-4)
    assert 'e' in uf.format_number(0.00001, precision=6)
```

## 3. Улучшение теста для конвертации температур (Мутант 4)

Проверить случай, когда только одна из единиц — температура, чтобы проверить логику `any()`.

*Листинг 22 — Код теста.*

```
def test_format_conversion_result_temperature_partial_match():
    # 'C' - температура, 'метр' - нет. Должно использовать символы температуры.
    result = uf.format_conversion_result(0, "c", 273.15, "метр")
    assert "0°C = 273.15K" not in result # Убеждаемся, что логика температур не
    смешивается с линейной

    # Этот тест проверяет, что блок IF выполняется, даже если одна из единиц имеет
    неизвестный символ

    # Мутационное тестирование показало, что блок IF должен быть чувствительным:

    # Добавим тест с неизвестным символом для убийства мутанта (any/all):
    result = uf.format_conversion_result(100, "цельсий", 273.15, "неизвестная_ед")
    # Должен сработать блок температур, поскольку "цельсий" есть в списке
    assert "°C" in result
    assert "неизвестная_ед" in result

    # Если бы было `all`, этот тест провалился бы. Следовательно, этот тест
    убивает мутанта.
```

## г) Модуль «validator»

Создание мутантов:

### 1. Мутации в `validate_numeric_input`

| Исходный код                 | Мутант (Ошибка)                                                  | Результат теста                                                                             | Убит?                                                                                     | Недостаток теста? |
|------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-------------------|
| try: return float(value_str) | try:<br>return int(value_str)<br>(Изменение функции конвертации) | test_validate_numeric_input_valid (для "3.14")<br>Провалится (ValueError или 3 вместо 3.14) | <br>Да | Нет               |
| raise ValueError(...)        | Удаление строки<br>(Удаление оператора)                          | test_validate_numeric_input_invalid Провалится<br>(тест ожидает исключение, но его нет)     | <br>Да | Нет               |

## 2. Мутации в validate\_positive\_number

| Исходный код          | Мутант (Ошибка)                         | Результат теста                                                                                   | Убит?                                                                                       | Недостаток теста? |
|-----------------------|-----------------------------------------|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|-------------------|
| if value < 0:         | if value <= 0:<br>(Изменение оператора) | test_validate_positive_number_valid (для 0) Провалится<br>(тест ожидает 0, но получит ValueError) | <br>Да | Нет               |
| raise ValueError(...) | Удаление строки<br>(Удаление оператора) | test_validate_positive_number_invalid Провалится                                                  | <br>Да | Нет               |

## 3. Мутации в validate\_temperature\_range

| Исходный код                                                  | Мутант (Ошибка)                                                             | Результат теста                                                                             | Убит ?                                                                                     | Недостаток теста? |
|---------------------------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|-------------------|
| <code>ranges.get(scale, (-float('inf'), float('inf')))</code> | <code>ranges.get(scale, (0, 0))</code><br>(Изменение значения по умолчанию) | Выживет                                                                                     | <br>Нет | Да                |
| <code>min_temp &lt;= value &lt;= max_temp</code>              | <code>min_temp &lt; value &lt;= max_temp</code><br>(Изменение оператора)    | <code>test_validate_temperature_range_valid</code> (для \$-273.15\$ и \$0\$K)<br>Провалится | <br>Да  | Нет               |
| <code>min_temp &lt;= value &lt;= max_temp</code>              | <code>min_temp &lt;= value</code><br>(Удаление части сравнения)             | Выживет                                                                                     | <br>Нет | Да                |

#### 4. Мутации в `validate_unit_exists` и `validate_category`

| Функция                           | Исходный код                                                   | Мутант (Ошибка)                                                                             | Результат теста                                                                          | Убит ? | Недостаток теста? |
|-----------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|--------|-------------------|
| <code>validate_unit_exists</code> | <code>if category in UNITS:</code><br>(Инверсия условия)       | <code>test_validate_unit_exists_valid</code> Провалится<br>(выдаст ошибку, когда не должен) |  Да | Нет    |                   |
| <code>validate_unit_exists</code> | <code>if unit in UNITS[category]:</code><br>(Инверсия условия) | <code>test_validate_unit_exists_invalid</code> (для "фут")<br>Провалится<br>(выдаст         |  Да | Нет    |                   |

| Функция              | Исходный код                                                | Мутант (Ошибка)                                                  | Результат теста                                                                         | Убит? | Недостаток теста? |
|----------------------|-------------------------------------------------------------|------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-------|-------------------|
|                      |                                                             | ошибку<br>KeyError<br>или<br>ValueError<br>(раньше<br>времени)   |                                                                                         |       |                   |
| validate_category    | if<br>category<br>in UNITS:<br>(Инверсия<br>условия)        | test_validate_category_invalid<br>Провалится                     |  Да  | Нет   |                   |
| validate_unit_exists | return<br>False<br>(Изменение<br>возвращаемого<br>значения) | test_validate_unit_exists_valid<br>Провалится<br>(False != True) |  Да | Нет   |                   |

Анализ их выживания:

Мутант 1 (Диапазон по умолчанию): Наш тест не включал случай неизвестной шкалы (например, rankine). Если изменить значение по умолчанию, тесты выживут. Мы добавили явную проверку, что неизвестная шкала не вызывает исключений.

Мутант 2 (Удаление верхней границы): Удаление проверки `value <= max_temp` позволило мутанту выжить, так как наши тесты не включали проверку верхних границ (например,  $10001^{\circ}\text{C}$ ).

Корректировка тестов:

1. Чтобы убить выжившего мутанта (Верхние границы температуры), мы добавили тесты, которые проверяют, что максимальные значения действительно являются границей, и что числа, превышающие их, вызывают ошибку.

Листинг 23 — Код теста.

```
def test_validate_temperature_range_upper_limit():
    # Проверка на максимальное значение (должно работать)
    assert val.validate_temperature_range(10000, 'c') == 10000

    # Проверка на значение выше максимума (должно провалиться)
    with pytest.raises(ValueError):
        val.validate_temperature_range(10000.0001, 'c') # Убивает мутанта с
        удалением верхней границы

    # Добавьте аналогичный тест для шкалы Фаренгейта (18032)
    with pytest.raises(ValueError):
        val.validate_temperature_range(18033, 'f')
```

2. Чтобы убить выжившего мутанта (Неизвестная шкала температуры), мы добавили тест, который проверяет поведение функции при передаче неизвестной шкалы.

Листинг 24 — Код теста.

```
def test_validate_temperature_range_unknown_scale():
    # Неизвестная шкала должна иметь бесконечный диапазон (-inf, inf)
    assert val.validate_temperature_range(10000000, 'rankine') == 10000000
    assert val.validate_temperature_range(-50000000, 'rankine') == -50000000 #
    Убивает мутанта с конечным диапазоном по умолчанию
```

#### д) Модуль «view»

Создание мутантов:

##### 1. Мутации в menu и history\_menu (Управление потоком)

| Функция | Исходный код                      | Мутант (Ошибка)                             | Результат теста                                              | Убит ?                                                                                      | Недостаток теста? |
|---------|-----------------------------------|---------------------------------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------|-------------------|
| menu    | if choice == 1:                   | if choice != 1:<br>(Инверсия условия)       | test_menu_navigation<br>Провалится (не вызовет convert_menu) | <br>Да | Нет               |
| menu    | try:<br>choice = int(input("> ")) | try:<br>choice = input(">> ")<br>(Удаление) | test_menu_navigation<br>Провалится (choice будет)            | <br>Да | Нет               |



| Функция                   | Исходный код                                          | Мутант (Ошибка)                                                            | Результат теста                                                                                                               | Убит ?                                                                                     | Недостаток теста?    |
|---------------------------|-------------------------------------------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|----------------------|
|                           |                                                       | <code>int()</code>                                                         | строкой, а не числом)                                                                                                         |                                                                                            |                      |
| <code>history_menu</code> | <code>elif choice == "0":<br/>return</code>           | <code>elif choice == "0":<br/>break</code><br>(Изменение оператора выхода) | Выживет (оба оператора прервут <code>while True</code> цикл)                                                                  | <br>Нет | Эквивалентный мутант |
| <code>history_menu</code> | <code>if not data:<br/>print("История пуста.")</code> | <code>if data:<br/>print("История пуста.")</code><br>(Инверсия условия)    | <code>test_show_recent_history</code><br>(должен печатать историю, но вместо этого напечатает "История пуста.")<br>Провалится | <br>Да | Нет                  |

## 2. Мутации в `convert_linear_menu`

| Исходный код                                                   | Мутант (Ошибка)                         | Результат теста | Убит ?                                                                                       | Недостаток теста? |
|----------------------------------------------------------------|-----------------------------------------|-----------------|----------------------------------------------------------------------------------------------|-------------------|
| <code>value = validator.validate_positive_number(value)</code> | Удаление строки<br>(Удаление валидации) | Выживет         | <br>Нет | Да                |
| <code>validator.validate_unit_exists(category, to_unit)</code> | Удаление строки<br>(Удаление валидации) | Выживет         | <br>Нет | Да                |
| <code>history.add_conversion(...)</code>                       | Удаление строки<br>(Удаление            | Выживет         | <br>Нет | Да                |

| Исходный код                               | Мутант (Ошибка)                                                          | Результат теста                                                                     | Убит ?                                                                                     | Недостаток теста? |
|--------------------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|-------------------|
|                                            | побочного эффекта)                                                       |                                                                                     |                                                                                            |                   |
| except Exception as e: print("Ошибка:", e) | except ValueError as e: print("Ошибка:", e)<br>(Сужение типа исключения) | test_convert_linear_menu_validation_error<br>Выживет (в тесте бросается ValueError) | <br>Нет | Да                |

### 3. Мутации в convert\_temperature\_menu

| Исходный код                                                      | Мутант (Ошибка)                                     | Результат теста                                                                                                 | Убит ?                                                                                       | Недостаток теста? |
|-------------------------------------------------------------------|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-------------------|
| if choice not in conversions_map:                                 | if choice in conversions_map:<br>(Инверсия условия) | test_convert_temperature_invalid_choice<br>Провалится (вызовет ошибку KeyError вместо print("Неверный выбор!")) | <br>Да  | Нет               |
| value = validator.validate_temperature_range(value, from_unit[0]) | Удаление строки<br>(Удаление валидации)             | Выживет                                                                                                         | <br>Нет | Да                |
| from_unit[0]                                                      | from_unit[1]<br>(Изменение индекса)                 | Выживет                                                                                                         | <br>Нет | Да                |

## Анализ их выживания:

Наш тест `test_convert_linear_menu_success` проверяет, что функции вызываются (`assert_called_once`), но неявно предполагает, что они успешны. Мутанты, удаляющие важные вызовы валидации или записи в историю, выживают, потому что:

1. Удаление `validator.validate_positive_number`: Тест использует `value=100`, что является положительным числом, и не проверяет вход с отрицательным числом.
2. Удаление `validator.validate_unit_exists`: Тест использует корректные единицы. Мутант выживает, потому что тест не проверяет, что при неверной единице должна произойти ошибка.

## Обнаруженные недостатки тестов в `convert_temperature_menu`:

1. Удаление `validator.validate_temperature_range`: Тест `test_convert_temperature_menu_success` использует `value=0`, что находится в безопасном диапазоне. Мутант, удаляющий эту валидацию, выживает, потому что тест не включает входное значение, которое находится вне диапазона (например, `-300C`).
2. Изменение индекса `from_unit[0]`: При использовании 'цельсий' (с - 0) или 'фаренгейт' (f - 0) в качестве `from_unit`, изменение индекса на [1] приведет к ошибке, только если второй символ не является ожидаемой шкалой. В нашем коде `from_unit` — это полные слова ('цельсий', 'фаренгейт' и т.д.). Использование `from_unit[0]` (первой буквы) для проверки диапазона очень хрупкое. Если мутант изменит это на `from_unit[1]`, тест может выжить, если `from_unit[1]` по совпадению соответствует ключу.

## Корректировка тестов:

1. Чтобы убить выживших мутантов в `convert_linear_menu` (Валидация),

мы проверили, что ввод отрицательного числа или несуществующей единицы вызывает ошибку, и что функция корректно ее обрабатывает (excerpt).

*Листинг 25 — Код теста.*

```
@patch('builtins.input')
@patch('builtins.print')
def test_convert_linear_menu_negative_value(mock_print, mock_input):
    # Ввод отрицательного числа
    mock_input.side_effect = ['-10', 'метр', 'километр']

    # Мы знаем, что validate_positive_number выдаст ValueError
    with patch('core.validator.validate_positive_number',
side_effect=ValueError("Ошибка: значение должно быть положительным")):
        view.convert_linear_menu('длина')
        # Проверяем, что была напечатана ошибка
        mock_print.assert_any_call("Ошибка:", 'Ошибка: значение должно быть
положительным') # Убивает мутанта с удаленной валидацией

@patch('builtins.input')
@patch('builtins.print')
def test_convert_linear_menu_invalid_unit(mock_print, mock_input):
    # Ввод существующего числа, но неверной единицы
    mock_input.side_effect = ['100', 'фут', 'метр']

    with patch('core.validator.validate_unit_exists',
side_effect=ValueError("Единица 'фут' не найдена")):
        view.convert_linear_menu('длина')
        # Проверяем, что была напечатана ошибка
        mock_print.assert_any_call("Ошибка:", "Единица 'фут' не найдена") #
Убивает мутанта с удаленной валидацией
```

2. Чтобы убить выжившего мутанта в `convert_temperature_menu` (Валидация), мы добавили проверку, что ввод температуры вне диапазона вызывает ошибку, и что эта ошибка также корректно обрабатывается.

*Листинг 26 — Код теста.*

```
@patch('builtins.input')
@patch('builtins.print')
def test_convert_temperature_menu_out_of_range(mock_print, mock_input):
    # Выбор Цельсий->Фаренгейт (1), ввод -300C (вне диапазона)
    mock_input.side_effect = ['1', '-300']

    with patch('core.validator.validate_temperature_range',
side_effect=ValueError("Температура должна быть в диапазоне")):
        view.convert_temperature_menu()
        # Проверяем, что была напечатана ошибка
        mock_print.assert_any_call("Ошибка:", 'Температура должна быть в
диапазоне') # Убивает мутанта с удаленной валидацией
```

## 3 АНАЛИЗ И ВЫВОДЫ

### 3.1 Оценка качества тестирования

Итоговое Качество Тестирования:

1. Полнота (Покрытие): Теперь тесты обеспечивают не только высокое покрытие строк кода, но и высокое покрытие логики. Почти все значимые операторы (математические, логические, сравнения) проверяются на их чувствительность к ошибкам.
2. Надежность (Чувствительность): Набор тестов способен обнаруживать даже атомарные, синтаксически небольшие ошибки (мутанты), которые могли бы быть допущены программистом, что является прямым доказательством его надежности.
3. Тестирование интеграционных точек: Особое внимание уделено интеграционным точкам (например, view вызывает validator и history). Проверка, что удаление валидации приводит к сбою, а удаление записи в историю обнаруживается, делает интеграционные тесты более надежными.

### 3.2 Анализ недостатков и их устранение

Улучшения, которые были достигнуты благодаря мутационному тестированию:

| Модуль         | Ранее Выжившие Мутанты (Пробелы)                                          | Статус после изменений                                                                     |
|----------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| convert_script | Формулы k_to_f и f_to_k не проверялись для всех ненулевых входных данных. | Убиты. Добавлены явные тесты для k_to_f и f_to_k, проверяющие ненулевые контрольные точки. |
| history        | Отсутствовала проверка перезаписи файла ('w' vs 'a') и                    | Убиты. Добавлены тесты, гарантирующие, что файл перезаписывается при сохранении, и         |

| Модуль                      | Ранее Выжившие Мутанты (Пробелы)                                                                | Статус после изменений                                                                                                                                                                                                                       |
|-----------------------------|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                             | проверки временных меток в статистике.                                                          | что поля <code>first_conversion</code> и <code>last_conversion</code> в статистике различаются и корректны.                                                                                                                                  |
| <code>unit_formatter</code> | Пропуски в проверке пограничных случаев форматирования чисел (научная нотация, конечная точка). | Убиты. Добавлены тесты, которые:<br>1) явно проверяют удаление конечной точки ( <code>5.0</code> $\rightarrow$ <code>5</code> ); 2) проверяют научную нотацию для пограничных значений ( <code>\$10^6\$</code> , <code>\$10^{-4}\$</code> ). |
| <code>validator</code>      | Недостаточно проверялись верхние границы температурных диапазонов и обработка неизвестных шкал. | Убиты. Добавлены тесты для верхних границ и для неизвестных шкал ( <code>rankine</code> ), что гарантирует корректную работу логики по умолчанию ( <code>-inf</code> , <code>+inf</code> ).                                                  |
| <code>view</code>           | Критический пробел: не проверялась корректная обработка исключений валидации.                   | Убиты. Добавлены тесты, имитирующие некорректный ввод (отрицательные числа, несуществующие единицы) и подтверждающие, что: 1) вызывается ошибка; 2) выводится сообщение об ошибке; 3) конвертация и запись в историю не происходят.          |

### 3.3 Итоговые выводы по результатам работы

В результате выполнения практической работы были достигнуты поставленные цели: освоены методы модульного и мутационного тестирования, разработаны и проанализированы тестовые наборы. В ходе исследования успешно применено мутационное тестирование, созданы эффективные модульные тесты с высоким покрытием кода. Мутационное тестирование позволило выявить и устранить пробелы в тестовом наборе, повысив качество проверки программного продукта. Приобретены практические навыки

разработки тестов, анализа покрытия кода и оценки эффективности тестовых сценариев. Полученные результаты подтверждают, что комбинированный подход к тестированию обеспечивает надёжную проверку качества программного обеспечения и помогает обнаруживать потенциальные ошибки на ранних этапах разработки.